Universidad de La Sabana

Métodos tabulares

Reinforcement Learning, segundo semestre

Ignacio Brevis

Versión 1.0. Si encuentras algún error o tienes comentarios, escríbeme a `ignaciobrve@unisabana.edu.co`.

Resumen

Esta guía trata el caso en el que el modelo **no se conoce** y el agente sólo dispone de muestras obtenidas al interactuar con el entorno. Presentamos cuatro algoritmos: **Monte Carlo**, que promedia retornos completos; **Sarsa**, que sustituye el retorno por una estimación *bootstrap*; **n-step Sarsa**, que interpola entre los dos anteriores; y **Q-learning**, que resuelve directamente la ecuación de optimalidad de Bellman. Al final se incluyen 10 ejercicios con pistas y soluciones completas.

Índice

1. Introducción: qué cambia sin modelo **2**
1.1. El problema . . . 2
1.2. Notación . . . 2
1.3. El gridworld . . . 3

2. Episodios, retornos y truncamiento **3**
2.1. El retorno . . . 3
2.2. Truncamiento . . . 4
2.3. El sesgo del truncamiento . . . 4

Parte I: Monte Carlo **5**

3. Monte Carlo: estimar promediando **5**
3.1. El estimador . . . 5
3.2. Por qué valores de acción y no de estado . . . 6
3.3. Exploración: por qué no basta con ser greedy . . . 6
3.4. La forma incremental . . . 7
3.5. El algoritmo . . . 7

Parte II: Diferencias temporales **8**

4. De Monte Carlo a TD: el bootstrapping **8**
4.1. La observación clave . . . 8
4.2. Las tres diferencias con Monte Carlo . . . 8

5. Sarsa **9**
5.1. La regla de actualización . . . 9
5.2. El algoritmo . . . 9

6. Sarsa de n pasos **10**
6.1. El retorno de n pasos . . . 10

7. Q-learning **11**
7.1. La regla de actualización 11
7.2. On-policy y off-policy 11
7.3. Sarsa y Q-learning sobre la misma transición 12

8. Visión unificada **12**

9. Convergencia **13**

10. Resumen **14**

11. Ejercicios **14**

12. Soluciones **17**

---

1. Introducción: qué cambia sin modelo

1.1. El problema

La ecuación de Bellman en forma matricial,

$$v = r_\pi + \gamma P_\pi v,$$

tiene solución cerrada $v = (I - \gamma P_\pi)^{-1} r_\pi$. Esa fórmula resuelve el problema de evaluación de una política *de una sola vez*. Pero depende de conocer dos objetos:

- la matriz P_π, es decir, la dinámica $p(s'|s,a)$ del entorno;
- el vector r_π, es decir, las recompensas esperadas $p(r|s,a)$.

Juntos, esos dos objetos constituyen el **modelo** del entorno. En la inmensa mayoría de los problemas reales el modelo no está disponible: un agente no recibe una tabla de probabilidades de transición al comenzar. Lo único que tiene es la capacidad de **interactuar y observar lo que ocurre**.

La idea en una frase. Un método **sin modelo** (*model-free*) sustituye las esperanzas que no se pueden calcular por **promedios de muestras** que sí puede observar.

Toda esta guía es un desarrollo de esa frase. La teoría necesaria ya está en la guía de probabilidad: si $\{x_1, \ldots, x_n\}$ son muestras independientes de una variable aleatoria X, entonces

$$\bar{x}_n = \frac{x_1 + x_2 + \cdots + x_n}{n} \longrightarrow \mathbb{E}[X],$$

y la convergencia es casi segura por la ley de los grandes números. Como en RL todo objeto de interés es una esperanza, todo objeto de interés es estimable por muestreo.

1.2. Notación

Mantenemos la notación de las guías anteriores y añadimos lo específico de esta.

Símbolo	Significa
$\mathcal{S},\ \mathcal{A}$	conjunto de estados y conjunto de acciones
$S_t,\ A_t,\ R_{t+1}$	estado, acción y recompensa del instante t (aleatorios)
$s,\ a,\ r$	valores concretos de los anteriores
G_t	retorno descontado a partir de t
$v_\pi(s)$	valor del estado s bajo la política π
$q_\pi(s,a)$	valor de la acción a en el estado s bajo π
$q_*(s,a)$	valor óptimo de acción
$\pi(a\|s)$	probabilidad de elegir a en s
γ	factor de descuento (aquí siempre 0.9)
α	tamaño de paso o tasa de aprendizaje
ε	parámetro de exploración de la política ε-greedy
T	longitud a la que truncamos los episodios
δ_t	error TD en el instante t
$\leftarrow$	asignación («el valor de la izquierda pasa a ser»)

1.3. El gridworld

Seguimos con el gridworld de 5×5 casillas de las guías anteriores.

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Recordemos las reglas: cinco acciones (arriba, derecha, abajo, izquierda, quedarse); movimiento exacto; chocar contra una pared cuesta -1 y deja al agente donde estaba; entrar en una casilla naranja cuesta -1; entrar en la azul da $+1$; las blancas dan 0. El factor de descuento es $\gamma = 0.9$.

Cuidado. Hay una diferencia esencial: anteriormente **conocíamos** el mapa y lo usábamos para construir P y r. Ahora el mapa se dibuja únicamente para que el lector pueda verificar las cuentas a mano. **El agente no ve el mapa completo.** Sólo observa la sucesión de estados, acciones y recompensas que va generando.

2. Episodios, retornos y truncamiento

La idea en una frase. Monte Carlo necesita retornos completos, y un retorno completo necesita que el episodio **termine**. Como nuestro gridworld no termina nunca, lo cortamos a T pasos.

2.1. El retorno

Recordemos la definición del retorno descontado:

$$G_t \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

y la de los valores que queremos estimar:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s], \qquad q_\pi(s,a) = \mathbb{E}[G_t|S_t = s,\, A_t = a].$$

Ambos son esperanzas, así que ambos son estimables promediando muestras de G_t. El problema es que G_t es una suma infinita: para obtener *una* muestra habría que esperar infinitos pasos.

2.2. Truncamiento

Nuestro gridworld es una tarea **continua**: la casilla azul no es terminal, y el agente sigue actuando indefinidamente. La solución práctica, consiste en cortar cada episodio a una longitud fija T y trabajar con el **retorno truncado**:

$$G_t^{(T)} \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_{t+T}.$$

Definición 2.1 (Episodio truncado)**.** Un *episodio truncado de longitud T* es una trayectoria

$$s_0,\ a_0,\ r_1,\ s_1,\ a_1,\ r_2,\ \ldots,\ s_{T-1},\ a_{T-1},\ r_T,\ s_T$$

generada siguiendo la política π desde un estado inicial s_0.

Ejemplo 2.2 (Un episodio truncado a $T = 5$)**.** Partimos de la casilla 23 y el agente elige, sucesivamente, izquierda, arriba, quedarse, quedarse, quedarse. Siguiendo el mapa:

t	**Estado** s_t	**Acción** a_t	**Llega a** s_{t+1}	**Recompensa** r_{t+1}
0	23	izquierda	22 (blanca)	0
1	22	arriba	17 (azul)	+1
2	17	quedarse	17 (azul)	+1
3	17	quedarse	17 (azul)	+1
4	17	quedarse	17 (azul)	+1

El retorno truncado desde $t = 0$ es

$$G_0^{(5)} = 0 + 0.9(1) + 0.9^2(1) + 0.9^3(1) + 0.9^4(1) = 3.0951.$$

2.3. El sesgo del truncamiento

Truncar no es gratis. El retorno verdadero desde $(23, \text{izquierda})$ con esta política es

$$q(23, \text{izquierda}) = 0 + \gamma(1 + \gamma + \gamma^2 + \cdots) = 0.9 \cdot \frac{1}{1 - 0.9} = 9,$$

mientras que el retorno truncado a cinco pasos vale 3.0951. La diferencia es enorme porque hemos descartado toda la cola de recompensas posteriores al paso 5.

Aumentar T reduce el sesgo:

T	$G_0^{(T)}$	Diferencia con el límite 9
5	3.0951	5.9049
10	5.5132	3.4868
20	7.7842	1.2158
50	8.9485	0.0515

Cuidado. El sesgo de truncamiento decae como γ^T, de modo que la T necesaria depende críticamente de γ. Con $\gamma = 0.9$ hace falta $T \approx 50$ para que el sesgo baje del 1 %; con $\gamma = 0.99$ harían falta unos 500 pasos. En la tabla de esta guía usamos $T = 5$ de manera ilustrativa: los valores numéricos que aparecen no deben leerse como estimaciones razonables de q_π.

Nota. Este problema es, de hecho, una motivación del *bootstrapping* Este método, que aprende a estimar $q(s_T, a_T)$ puede usar la estimación para reemplazar «la cola que el truncamiento descarta». Dicho de otro modo: los métodos de diferencias temporales no son sólo una alternativa a Monte Carlo, sino también la alternativa al sesgo que introduce el truncamiento.

Parte I: Monte Carlo

3. Monte Carlo: estimar promediando

La idea en una frase. Para estimar $q_\pi(s, a)$, generamos muchos episodios que empiecen por (s, a), calculamos el retorno de cada uno y los **promediamos**.

3.1. El estimador

Definición 3.1 (Estimador de Monte Carlo)**.** Sean $g^{(1)}, g^{(2)}, \ldots, g^{(n)}$ los retornos observados en n episodios que comienzan en el par (s, a) y siguen después la política π. El estimador de Monte Carlo es

$$q_n(s, a) \stackrel{\text{def}}{=} \frac{g^{(1)} + g^{(2)} + \cdots + g^{(n)}}{n}.$$

Por la ley de los grandes números, $q_n(s, a) \to q_\pi(s, a)$ casi seguramente cuando $n \to \infty$, siempre que los episodios sean independientes. Notemos que no hemos usado el modelo en ningún momento: sólo hemos sumado números observados.

Ejemplo 3.2 (Estimación por Monte Carlo en el gridworld)**.** Considerando una política aleatoria, estimemos $q_\pi(23, \text{izquierda})$ con tres episodios truncados a $T = 5$. En los tres, la primera acción es la misma (izquierda); lo que cambia es lo que el agente decide después.

Episodio A (el del Ejemplo 2.2): recompensas 0, +1, +1, +1, +1, con

$$g^{(A)} = 0 + 0.9 + 0.81 + 0.729 + 0.6561 = 3.0951.$$

Episodio B: $23 \to 22 \to 17 \to 12 \to 17 \to 17$, es decir, el agente llega al objetivo pero después sube a la casilla naranja 12 y vuelve a bajar. Recompensas 0, +1, −1, +1, +1:

$$g^{(B)} = 0 + 0.9(1) + 0.81(-1) + 0.729(1) + 0.6561(1) = 1.4751.$$

Episodio C: $23 \to 22 \to 21 \to 16 \to 11 \to 12$, en el que el agente se va hacia la izquierda y atraviesa tres casillas naranjas. Recompensas 0, −1, −1, 0, −1:

$$g^{(C)} = 0 + 0.9(-1) + 0.81(-1) + 0.729(0) + 0.6561(-1) = -2.3661.$$

El estimador de Monte Carlo con estos tres episodios es

$$q_3(23, \text{izquierda}) = \frac{3.0951 + 1.4751 - 2.3661}{3} = \frac{2.2041}{3} = 0.7347.$$

Observación. Los tres retornos son muy distintos entre sí (3.09, 1.48, −2.37) aunque la primera acción sea idéntica. Toda esa dispersión viene de las decisiones posteriores del agente, no del entorno, que es determinista. Ésta es la **varianza** característica de Monte Carlo: cada muestra acumula la aleatoriedad de todas las decisiones del episodio, y hacen falta muchos episodios para que el promedio se estabilice.

3.2. Por qué valores de acción y no de estado

En el caso en que conocemos el modelo, da igual trabajar con v o con q, porque se pasa de uno a otro con la dinámica. Sin modelo **no da igual**, y la razón se ve al intentar mejorar una política. La regla de mejora a partir de valores de estado es

$$\pi'(s) = \arg\max_a \Big[\underbrace{\textstyle\sum_r r\, p(r|s,a)}_{\text{hace falta el modelo}} + \gamma \underbrace{\textstyle\sum_{s'} p(s'|s,a)\, v_\pi(s')}_{\text{hace falta el modelo}} \Big],$$

y los dos términos marcados son justamente los que no tenemos. Conocer v_π no basta: saber lo buena que es cada casilla no dice qué acción lleva a cada casilla.

Con valores de acción, en cambio, la mejora es inmediata:

$$\pi'(s) = \arg\max_a \; q_\pi(s,a),$$

y aquí no aparece el modelo por ninguna parte. La comparación entre acciones ya está hecha dentro de q.

La idea en una frase. En RL sin modelo se estiman valores de **acción** porque son los únicos que permiten mejorar la política sin conocer la dinámica.

Nota. Esto explica una asimetría: en programación dinámica (con modelo) casi todo se escribe en términos de v, mientras que en RL sin modelo casi todo se escribe en términos de q. No es una preferencia estética. Es que q es la única de las dos funciones que, por sí sola, basta para actuar.

3.3. Exploración: por qué no basta con ser greedy

El estimador de Monte Carlo sólo puede estimar $q_\pi(s,a)$ para pares (s,a) que *aparecen* en los episodios. Si la política es determinista y greedy, en cada estado elige siempre la misma acción, de modo que las otras cuatro nunca se visitan y su valor nunca se estima. El agente queda atrapado en la primera acción que le pareció buena.

Hay dos maneras clásicas de resolverlo:

- **Inicios exploratorios** (*exploring starts*): obligar a que cada episodio empiece en un par (s,a) distinto, recorriéndolos todos. Es correcto en teoría, pero suele ser impracticable: exige poder colocar al agente en cualquier estado a voluntad, cosa imposible con un agente (robot) físico.
- **Políticas ε-greedy**: hacer que la política conserve siempre algo de aleatoriedad, de modo que todas las acciones tengan probabilidad positiva.

Definición 3.3 (Política ε-greedy)**.** Dada una estimación q y un parámetro $\varepsilon \in (0,1]$,

$$\pi(a|s) = \begin{cases} 1 - \dfrac{\varepsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1), & \text{si } a = \arg\max_{a'} q(s,a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|}, & \text{para las demás acciones.} \end{cases}$$

Ejemplo 3.4. En nuestro gridworld hay $|\mathcal{A}| = 5$ acciones. Con $\varepsilon = 0.5$, la acción greedy recibe probabilidad

$$1 - \tfrac{0.5}{5}(5-1) = 1 - 0.4 = 0.6,$$

y cada una de las otras cuatro recibe $0.5/5 = 0.1$.

Cuidado. Una política ε-greedy es un **compromiso**, no una solución. Explora lo suficiente para que todos los pares se visiten, pero por eso mismo *no* es óptima: con probabilidad ε toma deliberadamente una acción que cree peor. La política que se aprende es la mejor *entre las ε-greedy*, no la mejor en absoluto. Para recuperar la optimalidad hay que ir reduciendo ε hacia cero a lo largo del aprendizaje, lo que en la literatura se conoce como condición *GLIE* (*greedy in the limit with infinite exploration*).

3.4. La forma incremental

Guardar todos los retornos para promediarlos al final es un desperdicio de memoria. Afortunadamente, un promedio puede calcularse **sobre la marcha**. Si q_n es el promedio de los primeros n retornos, entonces

$$q_{n+1} = q_n + \frac{1}{n+1}\,(g^{(n+1)} - q_n). \tag{1}$$

Esta identidad, que el Ejercicio 2 pide demostrar, es mucho más importante de lo que parece. Léase así:

$$\underbrace{q_{n+1}}_{\text{nueva estimación}} = \underbrace{q_n}_{\text{actual}} + \underbrace{\alpha_n}_{\text{paso}}\,\big(\underbrace{g^{(n+1)}}_{\text{objetivo}} - \underbrace{q_n}_{\text{actual}}\big).$$

Observación. **Nueva estimación = estimación actual + paso × (objetivo − estimación actual).** Los cuatro algoritmos de esta guía tienen exactamente esta forma. Lo único que cambia de uno a otro es **qué se pone en el lugar del objetivo**.

Nota. La ecuación (1) es un caso particular del algoritmo de Robbins–Monro (que se vio en la guía de probabilidad) con $\alpha_n = 1/(n+1)$. Allí se vio que esa sucesión satisface las dos condiciones

$$\sum_n \alpha_n = \infty, \qquad \sum_n \alpha_n^2 < \infty,$$

y que bajo ellas el algoritmo converge casi seguramente a la media. Es decir: la convergencia de Monte Carlo *ya estaba demostrada*.

3.5. El algoritmo

Algoritmo 1 (Monte Carlo ε-greedy)**. Entrada:** γ, ε, longitud de truncamiento T.
Inicializar: $q(s,a)$ arbitrariamente para todo (s,a); π la política ε-greedy respecto de q.

Repetir para cada episodio:

1. Elegir un estado inicial s_0 y generar, siguiendo π, un episodio truncado

$$s_0,\, a_0,\, r_1,\, s_1,\, a_1,\, r_2,\, \ldots,\, s_{T-1},\, a_{T-1},\, r_T.$$

2. Inicializar $g \leftarrow 0$.
3. **Para** $t = T-1, T-2, \ldots, 0$ (recorriendo el episodio hacia atrás):
   (a) $g \leftarrow r_{t+1} + \gamma\, g$;

(b) $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[g - q(s_t, a_t)]$;

(c) actualizar $\pi(\cdot|s_t)$ como ε-greedy respecto de $q(s_t, \cdot)$.

Observación. El bucle recorre el episodio **hacia atrás** por eficiencia: calculando $g \leftarrow r_{t+1} + \gamma g$ de atrás hacia adelante se obtienen todos los retornos g_t del episodio en una sola pasada, en lugar de recalcular cada suma desde cero.

Parte II: Diferencias temporales

4. De Monte Carlo a TD: el bootstrapping

La idea en una frase. En lugar de esperar al final del episodio para conocer el retorno, **estimamos** con la recompensa inmediata más el valor que ya tenemos del estado siguiente. Aprender a partir de una estimación propia se conoce como *bootstrapping*.

4.1. La observación clave

Monte Carlo usa como objetivo el retorno completo g, que sólo se conoce al terminar el episodio. Pero la ecuación de Bellman nos dice que

$$q_\pi(s, a) = \mathbb{E}\big[R_{t+1} + \gamma\, q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s,\, A_t = a\big].$$

El lado derecho también es una esperanza, así que también se puede muestrear. Y la muestra correspondiente,

$$r + \gamma\, q(s', a'),$$

está disponible **inmediatamente después de una sola transición**, sin esperar al final de un episodio.

Definición 4.1 (Objetivo TD y error TD)**.** Dada una transición (s, a, r, s', a') y una estimación actual q, se definen

$$\underbrace{r + \gamma\, q(s', a')}_{\text{objetivo TD}}, \qquad \delta \stackrel{\text{def}}{=} \underbrace{r + \gamma\, q(s', a')}_{\text{objetivo}} - \underbrace{q(s, a)}_{\text{estimación}}.$$

La cantidad δ se llama *error de diferencia temporal* (o error TD).

El error TD mide la discrepancia entre lo que creíamos que valía (s, a) y lo que la experiencia de un paso sugiere que vale.

4.2. Las tres diferencias con Monte Carlo

	Monte Carlo	**Diferencias temporales**
Cuándo actualiza	al terminar el episodio	tras cada transición
Objetivo	g, el retorno observado	$r + \gamma q(s', a')$
Sesgo	insesgado (sin truncar)	sesgado, porque q es sólo una estimación
Varianza	alta: acumula todo el episodio	baja: sólo una transición
Tareas continuas	necesita truncar	funciona sin modificación

Cuidado. El compromiso entre sesgo y varianza es el eje de toda la comparación, y no hay un ganador universal. Monte Carlo no introduce sesgo pero sufre una varianza que crece con la longitud del episodio. TD introduce sesgo (porque arranca con un q inventado y aprende a partir de él) a cambio de una varianza mucho menor. En la práctica, la reducción de varianza suele compensar, y por eso los métodos TD dominan; pero la afirmación «TD es mejor que MC» es falsa en general.

5. Sarsa

La idea en una frase. Sarsa es la forma incremental de la Sección 3.4 con el objetivo TD en lugar del retorno. Su nombre es la lista de lo que necesita: **S**tate, **A**ction, **R**eward, **S**tate, **A**ction.

5.1. La regla de actualización

Definición 5.1 (Sarsa)**.** Tras observar la transición $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha\Big[\, r_{t+1} + \gamma\, q(s_{t+1}, a_{t+1}) - q(s_t, a_t) \,\Big]. \tag{2}$$

La acción a_{t+1} es la que el agente **realmente va a ejecutar** en el paso siguiente, elegida por su política ε-greedy. Este detalle, que parece menor, es exactamente lo que hace a Sarsa *on-policy*, y lo que lo separa de Q-learning.

Propiedad 5.2 (Qué ecuación resuelve Sarsa)**.** *Sarsa es el algoritmo de Robbins–Monro aplicado a la* ***Bellman expectation equation*** *para valores de acción,*

$$q_\pi(s, a) = \mathbb{E}\big[R_{t+1} + \gamma\, q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a\big].$$

Por tanto, con una política π fija, Sarsa estima q_π, el valor de esa *política.*

5.2. El algoritmo

Algoritmo 2 (Sarsa con política ε-greedy)**. Entrada:** γ, α, ε.
Inicializar: $q(s, a)$ arbitrariamente.

Repetir para cada episodio:

1. Elegir el estado inicial s_0 y la acción a_0 con la política ε-greedy respecto de q.
2. **Para** $t = 0, 1, 2, \ldots$:
   (a) ejecutar a_t, observar r_{t+1} y s_{t+1};
   (b) elegir a_{t+1} con la política ε-greedy respecto de $q(s_{t+1}, \cdot)$;
   (c) $q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha\big[r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)\big]$;
   (d) actualizar $\pi(\cdot|s_t)$ como ε-greedy respecto de $q(s_t, \cdot)$.

Observación. Los pasos (c) y (d) alternan **evaluación** y **mejora** de la política en cada transición. Ésta es la idea de *iteración de política generalizada*: no hace falta evaluar bien una política para empezar a mejorarla.

6. Sarsa de n pasos

La idea en una frase. Monte Carlo mira hasta el final del episodio; Sarsa mira un solo paso. Sarsa de n pasos mira n pasos y luego estima el resto. Los dos anteriores son los casos extremos.

6.1. El retorno de n pasos

Definición 6.1 (Retorno de n pasos)**.**

$$G_t^{(n)} \stackrel{\text{def}}{=} R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n q(S_{t+n}, A_{t+n}).$$

Los primeros n términos son recompensas *observadas*; el último es una *estimación* que sustituye a todo lo que vendría después. Variando n se recorre un espectro completo:

n	**Objetivo**	**Equivale a**
1	$r_{t+1} + \gamma q(s_{t+1}, a_{t+1})$	Sarsa
2	$r_{t+1} + \gamma r_{t+2} + \gamma^2 q(s_{t+2}, a_{t+2})$	—
$\vdots$	$\vdots$	
∞	$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$	Monte Carlo

La regla de actualización es la de siempre, con este objetivo:

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[G_t^{(n)} - q(s_t, a_t)].$$

Ejemplo 6.2 (Los retornos de n pasos en el gridworld)**.** Usemos el episodio A del Ejemplo 2.2, con las estimaciones actuales $q(22, \text{arriba}) = 4$ y $q(17, \text{quedarse}) = 5$. Las recompensas eran 0, +1, +1, +1, +1.

n	**Cálculo**	$G_0^{(n)}$
1	$0 + 0.9(4)$	3.6000
2	$0 + 0.9(1) + 0.81(5)$	4.9500
3	$0 + 0.9(1) + 0.81(1) + 0.729(5)$	5.3550
4	$0 + 0.9(1) + 0.81(1) + 0.729(1) + 0.6561(5)$	5.7195
5	$0 + 0.9(1) + 0.81(1) + 0.729(1) + 0.6561(1)$	3.0951

Cuidado. El último valor **rompe** la tendencia: con $n = 5$ el objetivo cae de 5.72 a 3.10. No es un error de cálculo. Con $n = 5$ agotamos el episodio truncado y ya no queda ningún $q(s_5, a_5)$ que añadir, de modo que el objetivo pasa a ser el retorno puro de Monte Carlo, que descarta toda la cola.
Esto ilustra lo que anunciábamos en la Sección 2.3: el término $\gamma^n q(s_{t+n}, a_{t+n})$ no es sólo un atajo, es lo que **repara el sesgo del truncamiento**. Mientras haya bootstrapping, la cola descartada se sustituye por una estimación de lo que valía; sin él, simplemente se pierde.

Observación (Cómo elegir n). No existe un valor universalmente bueno. Valores pequeños dan poco ruido pero propagan la información despacio: con $n = 1$, la noticia de que la casilla azul es valiosa tarda muchos episodios en llegar a las casillas lejanas, porque avanza una casilla por actualización. Valores grandes propagan rápido pero heredan la varianza de Monte Carlo. En la práctica, valores intermedios (n entre 4 y 8) suelen funcionar mejor que cualquiera de los dos extremos.

7. Q-learning

La idea en una frase. Q-learning cambia una sola cosa respecto de Sarsa: en lugar del valor de la acción que *se va a tomar*, usa el valor de la **mejor** acción disponible. Ese cambio mínimo lo convierte en un algoritmo de naturaleza distinta.

7.1. La regla de actualización

Definición 7.1 (Q-learning)**.** Tras observar la transición $(s_t, a_t, r_{t+1}, s_{t+1})$:

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha\Big[\, r_{t+1} + \gamma \max_a q(s_{t+1}, a) - q(s_t, a_t) \,\Big]. \tag{3}$$

Toda la diferencia con Sarsa, ecuación (2), está en un solo término:

$$\underbrace{q(s_{t+1}, a_{t+1})}_{\text{Sarsa}} \qquad \text{frente a} \qquad \underbrace{\max_a q(s_{t+1}, a)}_{\text{Q-learning}}.$$

Nótese además que la transición ya no incluye a_{t+1}, de modo que Q-learning **no necesita saber qué hará el agente a continuación**.

Propiedad 7.2 (Qué ecuación resuelve Q-learning)**.** *Q-learning es el algoritmo de Robbins–Monro aplicado a la* ***ecuación de optimalidad de Bellman****,*

$$q_*(s, a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a \Big].$$

Ésta es la diferencia de fondo. Sarsa resuelve la ecuación de Bellman (Bellman expectation), que depende de una política π concreta, y por eso estima q_π. Q-learning resuelve la ecuación de optimalidad, en cuyo enunciado **no aparece ninguna política**, y por eso estima directamente q_*.

7.2. On-policy y off-policy

Definición 7.3. En un algoritmo *on-policy*, la política que genera los datos (*behavior policy*) y la política que se evalúa o mejora (*target policy*) son la misma. En un algoritmo *off-policy*, pueden ser distintas.

Sarsa es on-policy porque su objetivo TD contiene $q(s_{t+1}, a_{t+1})$, y a_{t+1} sale de la política que el agente está usando: cambiar de política cambia el objetivo. Q-learning es off-policy porque su objetivo contiene $\max_a q(s_{t+1}, a)$, que no depende de cómo el agente elija sus acciones.

Nota. La consecuencia práctica es considerable: Q-learning puede aprender la política *óptima* mientras se comporta de cualquier otra manera. Puede aprender de datos antiguos, de datos generados por otro agente, o incluso de la actuación de un humano. Es lo que hace posible el *experience replay* de las redes Q profundas (DQN), donde las actualizaciones se

calculan sobre transiciones almacenadas hace miles de pasos, generadas por una política que ya no existe. Con Sarsa eso sería incorrecto.
La única condición que se mantiene es que la behavior policy de comportamiento debe **explorar lo suficiente**: cada par (s, a) tiene que visitarse infinitas veces, porque de los pares nunca visitados no se puede aprender nada.

7.3. Sarsa y Q-learning sobre la misma transición

La mejor forma de ver la diferencia es aplicar ambos a los mismos datos.

Ejemplo 7.4 (La misma transición, dos resultados opuestos)**.** El agente está en la casilla 22, toma la acción *arriba*, entra en la casilla azul 17 y recibe $r = +1$. Las estimaciones actuales son $q(22, \text{arriba}) = 4$ y, para la casilla 17:

Acción a	quedarse	abajo	arriba	izquierda	derecha
$q(17, a)$	5	3	2	2	2

La acción greedy en 17 es *quedarse*, con valor 5. Pero supongamos que la política ε-greedy, en su componente aleatoria, elige $a_{t+1} =$ *arriba*, cuyo valor es 2. Tomamos $\alpha = 0.1$ y $\gamma = 0.9$.

Sarsa usa la acción que realmente se tomará:

$$\text{objetivo} = 1 + 0.9\,(2) = 2.8, \qquad \delta = 2.8 - 4 = -1.2,$$

$$q(22, \text{arriba}) \leftarrow 4 + 0.1(-1.2) = 3.88.$$

Q-learning usa la mejor acción disponible:

$$\text{objetivo} = 1 + 0.9\,(5) = 5.5, \qquad \delta = 5.5 - 4 = 1.5,$$

$$q(22, \text{arriba}) \leftarrow 4 + 0.1(1.5) = 4.15.$$

Observación. Con exactamente los mismos datos, Sarsa **baja** la estimación y Q-learning la **sube**. La interpretación es transparente:

- Sarsa contabiliza el error que el agente está a punto de cometer. Para Sarsa, subir a la casilla 22 es menos atractivo de lo que se creía *porque el agente que de hecho actúa va a desperdiciar la oportunidad.*
- Q-learning supone que, a partir del estado siguiente, se actuará de forma óptima. Para Q-learning, subir a la casilla 22 es más atractivo de lo que se creía porque desde allí *se podría* obtener 5.

Ninguno de los dos está equivocado: responden a preguntas distintas. Sarsa responde «¿cuánto vale esto para el agente que soy?»; Q-learning, «¿cuánto valdría para el agente que podría llegar a ser?».

Cuidado. Esta diferencia tiene consecuencias de comportamiento, no sólo numéricas. Cuando existe una ruta óptima que pasa cerca de un riesgo (en nuestro mapa, la casilla azul 17 está rodeada de tres casillas naranjas), Sarsa aprende una política más **conservadora**, porque incorpora el coste de los errores exploratorios; Q-learning aprende la ruta óptima, que es la correcta *si se deja de explorar*, pero más costosa mientras se siga explorando. Es el fenómeno clásico que en la literatura se ilustra con el ejemplo del *cliff walking*.

8. Visión unificada

La idea en una frase. Los cuatro algoritmos son el mismo esquema de aproximación estocástica aplicado a cuatro ecuaciones distintas. Casi todas sus diferencias se deducen de cuál es esa ecuación.

Todos tienen la forma

$$q(s_t, a_t) \;\leftarrow\; q(s_t, a_t) + \alpha\big[\,\text{objetivo} - q(s_t, a_t)\,\big],$$

y se distinguen únicamente por el objetivo:

Algoritmo	**Objetivo**	**Ecuación que resuelve**	**Tipo**
Monte Carlo	g_t	$q_\pi(s,a) = \mathbb{E}[G_t \mid s, a]$	on-policy
Sarsa	$r_{t+1} + \gamma q(s_{t+1}, a_{t+1})$	Bellman expectation	on-policy
Sarsa de n pasos	$r_{t+1} + \cdots + \gamma^n q(s_{t+n}, a_{t+n})$	Bellman expectation, n pasos	"on-policy"
Q-learning	$r_{t+1} + \gamma \max_a q(s_{t+1}, a)$	**optimalidad** de Bellman	off-policy

De esta tabla se leen casi todas las propiedades que hemos ido discutiendo:

- Los tres primeros resuelven ecuaciones que mencionan una política π, luego estiman q_π y son on-policy. El cuarto resuelve una ecuación que no menciona ninguna política, luego estima q_* y es off-policy.
- Monte Carlo no contiene q dentro del objetivo: no hace bootstrapping, y por eso es insesgado pero de alta varianza.
- Sarsa de n pasos contiene q multiplicado por γ^n: cuanto mayor es n, menos pesa la estimación y más pesan los datos. De ahí que n sea la conexión entre MC y Sarsa.

Observación (Una quinta variante)*.* La tabla sugiere de inmediato una casilla que no hemos rellenado: ¿y si en lugar de $q(s_{t+1}, a_{t+1})$ (Sarsa) o de $\max_a q(s_{t+1}, a)$ (Q-learning) usáramos $\sum_a \pi(a|s_{t+1})\, q(s_{t+1}, a)$, es decir, el valor *esperado* bajo la política? Ese algoritmo existe, se llama **Expected Sarsa**, y elimina la varianza debida a la elección concreta de a_{t+1} a cambio de algo más de cómputo. El Ejercicio 8 lo calcula numéricamente.

9. Convergencia

Esta sección enuncia los resultados; las herramientas para demostrarlos (Robbins–Monro, teorema de Dvoretzky, convergencia casi segura) están en la guía de probabilidad, y las demostraciones completas, en los Capítulos 6 y 7 de Zhao.

Teorema 9.1 (Condiciones comunes)**.** *Todos los algoritmos de esta guía requieren:*

(a) ***Visitas infinitas****: cada par (s, a) se visita infinitas veces;*

(b) ***Tamaños de paso admisibles****:*

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty.$$

La condición (b) es exactamente la que se analizó en la guía de probabilidad: la primera suma impide que los pasos se encojan demasiado deprisa, y la segunda obliga a que se encojan lo suficiente para que el ruido se promedie. La sucesión $\alpha_k = 1/k$ las cumple; un paso constante no cumple la segunda.

Teorema 9.2 (Convergencia de Q-learning)**.** *Bajo las condiciones anteriores, y si las recompensas tienen varianza acotada,*

$$q_k \xrightarrow{c.s.} q_*,$$

con independencia de la política de comportamiento *que haya generado los datos, siempre que ésta explore lo suficiente para garantizar (a).*

Teorema 9.3 (Convergencia de Sarsa)**.** *Bajo las mismas condiciones, Sarsa converge casi seguramente a q_π si la política π se mantiene fija. Para que converja a q_* hace falta además que la política ε-greedy sea* GLIE*, es decir, que $\varepsilon_k \to 0$ de modo que la política se vuelva greedy en el límite sin dejar de explorar por el camino.*

Cuidado. La asimetría entre los dos teoremas no es un tecnicismo. Q-learning converge a q_* sin condiciones sobre la política; Sarsa necesita que la política converja a política *greedy*. La razón vuelve a estar en la tabla de la sección anterior: el punto fijo de Sarsa *es* q_π, de modo que si π no tiende a la política óptima, el límite de Sarsa tampoco.

Observación. Conviene recordar que todos estos resultados son *asintóticos* y para el caso **tabular** (en el que se guarda un número por cada par (s, a)). Con aproximación de funciones (redes neuronales, por ejemplo) las garantías se debilitan notablemente, y en el caso off-policy pueden perderse por completo: la combinación de bootstrapping, aproximación y off-policy se conoce como la *tríada mortal* y puede divergir.

10. Resumen

Concepto	**Expresión o idea**
Esquema común	nueva = actual + α (objetivo − actual)
Retorno truncado	$G_t^{(T)} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{T-1} r_{t+T}$
Monte Carlo	objetivo = g_t; insesgado, alta varianza
Por qué q y no v	$\pi'(s) = \arg\max_a q(s,a)$ no necesita modelo
ε-greedy	greedy: $1 - \varepsilon(\lvert\mathcal{A}\rvert - 1)/\lvert\mathcal{A}\rvert$; resto: $\varepsilon/\lvert\mathcal{A}\rvert$
Error TD	$\delta_t = r_{t+1} + \gamma q(s_{t+1}, a_{t+1}) - q(s_t, a_t)$
Sarsa	objetivo = $r_{t+1} + \gamma q(s_{t+1}, a_{t+1})$
Sarsa de n pasos	objetivo = $r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n q(s_{t+n}, a_{t+n})$
Q-learning	objetivo = $r_{t+1} + \gamma \max_a q(s_{t+1}, a)$
On-policy / off-policy	¿aparece la política en el objetivo?
Condiciones de paso	$\sum_k \alpha_k = \infty$, $\sum_k \alpha_k^2 < \infty$

11. Ejercicios

Cada ejercicio incluye una *pista*. Las soluciones completas están en la Sección 12. Se recomienda intentar el ejercicio antes de leer la pista, y la pista antes de leer la solución. En todos ellos, $\gamma = 0.9$ y el mapa es el de la Sección 1.3.

Ejercicio 1 (Retornos truncados)**.** El agente parte de la casilla 24 y ejecuta: arriba, arriba, izquierda, izquierda, abajo.

(a) Reconstruya el episodio: a qué casilla llega en cada paso y qué recompensa recibe.

(b) Calcule $G_0^{(5)}$.

(c) Calcule $G_2^{(3)}$, es decir, el retorno truncado desde $t = 2$.

Pista: la casilla 24 está en la fila 4, columna 4. Vaya paso a paso sobre el mapa y anote el color de cada casilla de llegada antes de calcular nada.

Ejercicio 2 (Forma incremental)**.** Sea $q_n = (g^{(1)} + \cdots + g^{(n)})/n$ el promedio de los primeros n retornos.

(a) Demuestre la identidad (1): $q_{n+1} = q_n + \frac{1}{n+1}(g^{(n+1)} - q_n)$.

(b) Verifíquela numéricamente con los tres retornos del Ejemplo 3.2, partiendo de $q_1 = 3.0951$.

(c) ¿Qué ventaja práctica tiene esta forma frente a guardar todos los retornos?

Pista: para (a), escriba $q_{n+1} = \frac{1}{n+1}\big(\sum_{i=1}^{n} g^{(i)} + g^{(n+1)}\big)$ y sustituya $\sum_{i=1}^{n} g^{(i)} = n\, q_n$.

Ejercicio 3 (Por qué no bastan los valores de estado)**.** Suponga que un oráculo le entrega los valores *exactos* $v_\pi(s)$ de las 25 casillas, pero no le dice nada sobre la dinámica del entorno.

(a) ¿Puede determinar qué acción tomar en la casilla 22? Justifique.

(b) Suponga ahora que le entregan $q_\pi(22, a)$ para las cinco acciones. ¿Puede determinarla?

(c) Enuncie en una frase la conclusión general.

Pista: escriba la regla de mejora de política en ambos casos y señale, en cada una, qué cantidades necesita que no tiene.

Ejercicio 4 (Monte Carlo a mano)**.** Estimamos $q_\pi(22, \text{arriba})$ con dos episodios truncados a $T = 4$, ambos comenzando con la acción *arriba* desde la casilla 22:

- **Episodio 1:** $22 \to 17 \to 17 \to 17 \to 17$ (quedarse las tres veces).
- **Episodio 2:** $22 \to 17 \to 18 \to 17 \to 17$ (derecha, izquierda, quedarse).

(a) Anote las recompensas de cada episodio.

(b) Calcule los dos retornos.

(c) Calcule el estimador de Monte Carlo.

(d) Compare con el valor verdadero $q_\pi(22, \text{arriba}) = 10$ bajo la política que siempre se queda en la azul, y explique las dos fuentes de la discrepancia.

Pista: para (d), una de las fuentes es el truncamiento y la otra es que el episodio 2 no sigue esa política.

Ejercicio 5 (Un paso de Sarsa)**.** El agente está en la casilla 23, toma la acción *izquierda* y llega a la casilla 22. En el paso siguiente su política ε-greedy elige *arriba*. Las estimaciones actuales son $q(23, \text{izquierda}) = 2$ y $q(22, \text{arriba}) = 4$. Use $\alpha = 0.2$.

(a) ¿Cuál es la recompensa r de esa transición?

(b) Calcule el objetivo TD y el error TD.

(c) Calcule el nuevo valor de $q(23, \text{izquierda})$.

(d) ¿Qué habría pasado si α fuera 1? ¿Y si fuera 0?

Pista: la casilla 22 es blanca, de modo que entrar en ella no da recompensa.

Ejercicio 6 (Sarsa frente a Q-learning)**.** El agente está en la casilla 13, toma la acción *abajo* y entra en la casilla 18, que es naranja. Las estimaciones para la casilla 18 son:

Acción a	izquierda	arriba	derecha	abajo	quedarse
$q(18, a)$	6	1	0	2	-1

Además, $q(13, \text{abajo}) = 3$ y $\alpha = 0.5$. La política ε-greedy elige como acción siguiente *quedarse*.

(a) Calcule la actualización de Sarsa.

(b) Calcule la actualización de Q-learning.

(c) Explique el signo de cada error TD en términos del mapa.

(d) ¿En qué situación darían ambos el mismo resultado?

Pista: para (c), observe hacia dónde lleva la acción *izquierda* desde la casilla 18, y compárelo con *quedarse*.

Ejercicio 7 (Retornos de n pasos)**.** Considere el episodio $22 \to 17 \to 22 \to 17 \to 17$ generado por las acciones arriba, abajo, arriba, quedarse, con las estimaciones $q(17, \text{abajo}) = 3$, $q(22, \text{arriba}) = 4$ y $q(17, \text{quedarse}) = 5$.

(a) Anote las recompensas del episodio.

(b) Calcule $G_0^{(1)}$, $G_0^{(2)}$ y $G_0^{(3)}$.

(c) ¿Cuál de los tres objetivos depende más de las estimaciones actuales y cuál de los datos observados?

Pista: el peso de la estimación en $G_0^{(n)}$ es γ^n. Calcule 0.9, 0.81 y 0.729 y compárelos.

Ejercicio 8 (Expected Sarsa)**.** Con los valores $q(17, \cdot)$ del Ejemplo 7.4 (quedarse 5, abajo 3, arriba 2, izquierda 2, derecha 2) y una política ε-greedy con $\varepsilon = 0.5$ sobre $|\mathcal{A}| = 5$ acciones:

(a) Escriba la probabilidad de cada una de las cinco acciones.

(b) Calcule $\sum_a \pi(a|17)\, q(17, a)$.

(c) Compárelo con los dos valores que usan Sarsa y Q-learning en ese mismo ejemplo (2 y 5 respectivamente) y explique la relación entre los tres números.

Pista: para (c), piense en qué es un promedio ponderado respecto del máximo y respecto de una muestra individual.

Ejercicio 9 (Tamaños de paso)**.** Un estudiante propone entrenar Q-learning con $\alpha_k = 1/\sqrt{k}$ y otro con $\alpha_k = 0.1$ constante.

(a) Verifique cuáles de las dos condiciones del Teorema de convergencia cumple cada propuesta.

(b) ¿Cuál de las dos garantiza convergencia casi segura a q_*?

(c) Pese a su respuesta anterior, el paso constante es el más usado en la práctica. Dé una razón.

Pista: recuerde que $\sum_k k^{-p}$ converge si y sólo si $p > 1$. Este ejercicio reutiliza directamente el que ya se resolvió en la guía de probabilidad.

Ejercicio 10 (Clasificación)**.** Para cada una de las afirmaciones siguientes, diga si es verdadera o falsa y justifique brevemente.

(a) Monte Carlo puede aplicarse sin cambios a una tarea continua.

(b) Q-learning puede aprender de episodios generados por una política completamente aleatoria.

(c) Sarsa con ε fijo converge a q_*.

(d) Sarsa de n pasos con $n = 1$ es exactamente Sarsa.

(e) Si el entorno es determinista, el objetivo de Monte Carlo tiene varianza cero.

Pista: para (e), recuerde de dónde sale la aleatoriedad en nuestro gridworld.

12. Soluciones

Solución 1.

(a) La casilla 24 está en $(4, 4)$. El episodio es:

t	Estado	Acción	Llega a	r_{t+1}
0	24	arriba	19 (blanca)	0
1	19	arriba	14 (blanca)	0
2	14	izquierda	13 (blanca)	0
3	13	izquierda	12 (naranja)	-1
4	12	abajo	17 (azul)	$+1$

(b)
$$G_0^{(5)} = 0 + 0.9(0) + 0.81(0) + 0.729(-1) + 0.6561(1) = -0.0729.$$

(c) Desde $t = 2$, las tres recompensas son $r_3 = 0$, $r_4 = -1$, $r_5 = +1$:
$$G_2^{(3)} = 0 + 0.9(-1) + 0.81(1) = -0.09.$$

Obsérvese que $G_0^{(5)}$ es casi cero pese a que el episodio termina en el objetivo: el descuento reduce el $+1$ final a 0.6561, y el -1 del paso anterior lo cancela casi por completo. Llegar tarde a una recompensa vale muy poco.

Solución 2.

(a) Por definición,
$$q_{n+1} = \frac{1}{n+1}\sum_{i=1}^{n+1} g^{(i)} = \frac{1}{n+1}\Big(\sum_{i=1}^{n} g^{(i)} + g^{(n+1)}\Big) = \frac{1}{n+1}\big(n\,q_n + g^{(n+1)}\big),$$
donde hemos usado $\sum_{i=1}^{n} g^{(i)} = n\,q_n$. Sumando y restando q_n:
$$q_{n+1} = \frac{n\,q_n + g^{(n+1)}}{n+1} = \frac{(n+1)q_n - q_n + g^{(n+1)}}{n+1} = q_n + \frac{1}{n+1}\big(g^{(n+1)} - q_n\big).$$

(b) Con $q_1 = 3.0951$ y $g^{(2)} = 1.4751$:
$$q_2 = 3.0951 + \tfrac{1}{2}(1.4751 - 3.0951) = 3.0951 - 0.81 = 2.2851.$$
Con $g^{(3)} = -2.3661$:
$$q_3 = 2.2851 + \tfrac{1}{3}(-2.3661 - 2.2851) = 2.2851 - 1.5504 = 0.7347.$$
Coincide con el promedio directo calculado en el Ejemplo 3.2.

(c) Memoria constante: basta guardar q_n y el contador n, en lugar de la lista completa de retornos. Además permite usar la estimación *en cualquier momento*, sin esperar a reunir todas las muestras, que es justo lo que necesita un agente que debe actuar mientras aprende.

Solución 3.

(a) **No.** La regla de mejora a partir de valores de estado es

$$\pi'(22) = \arg\max_a \Big[\textstyle\sum_r r\, p(r|22,a) + \gamma \sum_{s'} p(s'|22,a) v_\pi(s') \Big],$$

y requiere saber a qué casilla lleva cada acción y qué recompensa produce. Sin modelo, conocer $v_\pi(17) = 10$ no sirve de nada, porque no sabemos *cuál de las cinco acciones* lleva a la casilla 17.

(b) **Sí.** Basta con $\pi'(22) = \arg\max_a q_\pi(22, a)$: se comparan los cinco números y se toma el mayor. No interviene ninguna cantidad desconocida.

(c) Sin modelo, los valores de estado no bastan para actuar, y los valores de acción sí. Por eso los métodos libres de modelo estiman q y no v.

Solución 4.

(a) **Episodio 1:** entra en la azul y se queda: recompensas $+1, +1, +1, +1$.
Episodio 2: entra en la azul $(+1)$, va a la derecha a la casilla 18, que es naranja (-1), vuelve a la izquierda a la azul $(+1)$ y se queda $(+1)$: recompensas $+1, -1, +1, +1$.

(b)
$$g^{(1)} = 1 + 0.9 + 0.81 + 0.729 = 3.4390,$$
$$g^{(2)} = 1 + 0.9(-1) + 0.81(1) + 0.729(1) = 1 - 0.9 + 0.81 + 0.729 = 1.6390.$$

(c)
$$q_2(22, \text{arriba}) = \frac{3.4390 + 1.6390}{2} = \frac{5.0780}{2} = 2.5390.$$

(d) La estimación 2.539 está muy por debajo de 10, por dos razones distintas:

- **Truncamiento**: aun con la política perfecta, el episodio 1 sólo acumula cuatro recompensas y da 3.439 en lugar de 10. El resto se ha descartado.
- **La política muestreada no es la óptima**: el episodio 2 incluye una excursión a una casilla naranja, de modo que estima el valor de una política peor. Esto no es un defecto del estimador: Monte Carlo estima correctamente q_π para la política π *que generó los datos*, y esa política no es la que siempre se queda en la azul.

Solución 5.

(a) La casilla 22 es blanca, luego $r = 0$.

(b) Objetivo TD:
$$r + \gamma\, q(22, \text{arriba}) = 0 + 0.9(4) = 3.6.$$
Error TD:
$$\delta = 3.6 - q(23, \text{izquierda}) = 3.6 - 2 = 1.6.$$

(c)
$$q(23, \text{izquierda}) \leftarrow 2 + 0.2(1.6) = 2 + 0.32 = 2.32.$$

(d) Con $\alpha = 1$ la estimación saltaría directamente al objetivo, $q(23, \text{izquierda}) = 3.6$, descartando por completo lo aprendido antes: el agente creería ciegamente a la última muestra. Con $\alpha = 0$ no habría actualización alguna y el agente no aprendería nunca. El tamaño de paso regula cuánta confianza se concede a cada muestra nueva frente al historial acumulado.

Solución 6. La casilla 18 es naranja, luego $r = -1$.

(a) **Sarsa** usa la acción elegida, *quedarse*, con $q(18, \text{quedarse}) = -1$:

$$\text{objetivo} = -1 + 0.9(-1) = -1.9, \qquad \delta = -1.9 - 3 = -4.9,$$

$$q(13, \text{abajo}) \leftarrow 3 + 0.5(-4.9) = 3 - 2.45 = 0.55.$$

(b) **Q-learning** usa la mejor acción, *izquierda*, con $q(18, \text{izquierda}) = 6$:

$$\text{objetivo} = -1 + 0.9(6) = 4.4, \qquad \delta = 4.4 - 3 = 1.4,$$

$$q(13, \text{abajo}) \leftarrow 3 + 0.5(1.4) = 3 + 0.7 = 3.7.$$

(c) Desde la casilla 18, la acción *izquierda* lleva a la casilla 17, la azul: es la salida buena de una casilla mala, y de ahí que $q(18, \text{izquierda}) = 6$ sea el mayor valor de la fila. La acción *quedarse*, en cambio, mantiene al agente en una casilla naranja cobrando -1 indefinidamente.

Sarsa da un error negativo grande porque contabiliza las dos malas noticias: se ha entrado en una casilla naranja *y* el agente va a quedarse en ella. Q-learning da un error positivo porque, aunque entrar costó -1, desde la casilla 18 se puede salir de inmediato hacia el objetivo, y eso es lo que el máx supone que ocurrirá.

(d) Coincidirían si la acción que la política elige fuera precisamente la codiciosa, es decir, si $a_{t+1} = \arg\max_a q(s_{t+1}, a)$. Esto sucede siempre que la exploración no se active, y en el límite cuando $\varepsilon \to 0$. De ahí que Sarsa y Q-learning tiendan a parecerse cuanto menor es la exploración.

Solución 7.

(a) El episodio es $22 \to 17$ (azul, $+1$), $17 \to 22$ (blanca, 0), $22 \to 17$ (azul, $+1$), $17 \to 17$ (azul, $+1$). Las recompensas son $+1$, 0, $+1$, $+1$.

(b) Los pares (s_t, a_t) relevantes son $(17, \text{abajo})$ en $t = 1$, $(22, \text{arriba})$ en $t = 2$ y $(17, \text{quedarse})$ en $t = 3$:

$$G_0^{(1)} = 1 + 0.9\, q(17, \text{abajo}) = 1 + 0.9(3) = 3.7,$$

$$G_0^{(2)} = 1 + 0.9(0) + 0.81\, q(22, \text{arriba}) = 1 + 0.81(4) = 4.24,$$

$$G_0^{(3)} = 1 + 0.9(0) + 0.81(1) + 0.729\, q(17, \text{quedarse}) = 1.81 + 0.729(5) = 5.455.$$

(c) El peso de la estimación es γ^n: vale 0.9 para $n = 1$, 0.81 para $n = 2$ y 0.729 para $n = 3$. Por tanto $G_0^{(1)}$ es el que más depende de las estimaciones actuales y $G_0^{(3)}$ el que más depende de los datos observados. Esta es exactamente la perilla que n controla: a mayor n, más peso tienen los datos reales y menos el *bootstrapping*, acercándose a Monte Carlo.

Solución 8.

(a) La acción greedy es *quedarse* (valor 5). Con $\varepsilon = 0.5$ y $|\mathcal{A}| = 5$:

$$\pi(\text{quedarse}|17) = 1 - \tfrac{0.5}{5}(4) = 0.6,$$

y cada una de las otras cuatro recibe $0.5/5 = 0.1$. Comprobación: $0.6 + 4(0.1) = 1$.

(b)

$$\sum_a \pi(a|17)\, q(17, a) = 0.6(5) + 0.1(3) + 0.1(2) + 0.1(2) + 0.1(2) = 3.0 + 0.3 + 0.6 = 3.9.$$

(c) Los tres números son 2 (Sarsa, en la realización concreta del ejemplo), 3.9 (Expected Sarsa) y 5 (Q-learning), y la relación entre ellos es sistemática:

- Q-learning usa el **máximo**, 5, que es una cota superior de cualquier promedio ponderado.
- Expected Sarsa usa el **promedio** bajo la política, 3.9, que está necesariamente entre el mínimo y el máximo de la fila.
- Sarsa usa **una muestra** de esa misma distribución. Su valor esperado es 3.9, el mismo que Expected Sarsa, pero cualquier realización concreta puede ser 5, 3 o 2. En este caso salió 2.

Es decir, Sarsa y Expected Sarsa tienen el mismo objetivo *en esperanza*; Expected Sarsa simplemente elimina la varianza debida al sorteo de a_{t+1} calculando el promedio en lugar de muestrearlo. Q-learning, en cambio, apunta a un objetivo distinto.

Solución 9.

(a) Con $\alpha_k = 1/\sqrt{k}$, es decir $p = 1/2$:

$$\sum_k \tfrac{1}{\sqrt{k}} = \infty \ \checkmark, \qquad \sum_k \tfrac{1}{k} = \infty \ \times .$$

Cumple la primera condición pero no la segunda. Con $\alpha_k = 0.1$ constante:

$$\sum_k 0.1 = \infty \ \checkmark, \qquad \sum_k 0.01 = \infty \ \times .$$

Tampoco cumple la segunda.

(b) **Ninguna de las dos.** La propuesta admisible más sencilla sería $\alpha_k = 1/k$, que cumple ambas. En general $\alpha_k = k^{-p}$ sirve exactamente para $1/2 < p \leq 1$.

(c) Porque un paso que decrece a cero hace que el agente deje de aprender. Si el entorno cambia, o si la política de comportamiento evoluciona (como ocurre siempre que ε se va reduciendo), un agente con $\alpha_k \to 0$ no puede adaptarse: ya ha congelado sus estimaciones. El paso constante mantiene una varianza residual proporcional a α a cambio de conservar plasticidad indefinidamente. Se sacrifica la convergencia asintótica, que es una propiedad del límite, por la capacidad de seguir aprendiendo, que es una propiedad útil en todo instante finito.

Solución 10.

(a) **Falsa.** Monte Carlo necesita el retorno completo, y en una tarea continua el episodio no termina nunca. Hay que truncar, lo que introduce sesgo, como vimos en la Sección 2.3.

(b) **Verdadera.** Es la propiedad que define a un algoritmo off-policy: el objetivo no depende de cómo se eligieron las acciones, sino sólo del máximo sobre el estado siguiente. La única exigencia es que la política aleatoria visite todos los pares (s, a) infinitas veces, cosa que una política uniforme cumple.

(c) **Falsa.** Con ε fijo, Sarsa converge a q_π donde π es la política ε-greedy, que no es óptima porque toma acciones subóptimas con probabilidad ε. Para alcanzar q_* hace falta $\varepsilon_k \to 0$ (condición GLIE).

(d) **Verdadera.** Sustituyendo $n = 1$ en la definición del retorno de n pasos se obtiene $G_t^{(1)} = r_{t+1} + \gamma q(s_{t+1}, a_{t+1})$, que es exactamente el objetivo de Sarsa.

(e) **Falsa.** En nuestro gridworld el entorno es determinista, pero la política no lo es: toda la aleatoriedad del retorno proviene de las decisiones del agente. El Ejemplo 3.2 lo muestra con tres retornos muy distintos (3.09, 1.48, -2.37) producidos por la misma acción inicial en un entorno sin ningún azar. La varianza sólo sería cero si además la política fuera determinista.